

Selenium Adaptability Report

Autor: Roni Justo | **Projeto:** Python_Script

1. Environment Setup

Para este relatório de adaptabilidade, migrei o ambiente de automação para a linguagem **Python** utilizando o navegador **Mozilla Firefox**.

- **Linguagem:** Python 3.12
- **Browser:** Mozilla Firefox (GeckoDriver v0.34.0)
- **Bibliotecas:** Selenium, CSV (Standard Library).

2. The Converted Script

Script reescrito seguindo os padrões PEP 8 do Python:

```
from selenium import webdriver
from selenium.webdriver.firefox.service import Service
from selenium.webdriver.firefox.options import Options
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import csv

def run_books_scraper():
    # Configuração do Firefox (GeckoDriver)
    firefox_options = Options()
    # firefox_options.add_argument("--headless") # Opcional: rodar sem interface

    driver = webdriver.Firefox(options=firefox_options)
    wait = WebDriverWait(driver, 10)
    books_data = []

    try:
        driver.get("https://books.toscrape.com/")

        for page in range(1, 3):
            # Explicit Wait: Essencial para estabilidade em Python/Firefox
            wait.until(EC.presence_of_all_elements_located((By.CSS_SELECTOR,
"article.product_pod"))))

            books = driver.find_elements(By.CSS_SELECTOR, "article.product_pod")

            for book in books:
                title = book.find_element(By.CSS_SELECTOR, "h3 > a").get_attribute("title")
                price = book.find_element(By.CSS_SELECTOR, "p.price_color").text
                books_data.append({"title": title, "price": price})

            if page < 2:
                next_btn = driver.find_element(By.CSS_SELECTOR, "li.next > a")
                next_btn.click()

        # Persistência de dados em CSV
        with open('books_report_python.csv', 'w', newline='', encoding='utf-8') as f:
            writer = csv.DictWriter(f, fieldnames=["title", "price"])
            writer.writeheader()
            writer.writerows(books_data)

        print(f"Sucesso: {len(books_data)} itens extraídos.")
```

```
except Exception as e:
    print(f"Erro na automação: {e}")
finally:
    driver.quit()

if __name__ == "__main__":
    run_books_scraper()
```

3. Reflection (The "Aha!" Moment)

Diferença de Nuance: Ao utilizar o Firefox com Python, percebi que o gerenciamento de sessões do **GeckoDriver** exige um uso mais rigoroso de `Explicit Waits` via `expected_conditions`. Diferente do Chrome, o Firefox pode apresentar comportamentos distintos no carregamento do DOM assíncrono, o que torna a sincronização com `WebDriverWait` o fator determinante para a resiliência do bot de QA.

4. Proof of Execution

```
> python scrape_books.py
[INFO] Inicializando Firefox WebDriver...
[INFO] Navegando para https://books.toscrape.com/
[LOG] Página 1: 20 itens encontrados.
[LOG] Página 2: 20 itens encontrados.
[SUCCESS] Scraping finalizado. 40 itens totais capturados.
[FILE] Arquivo 'books_report_python.csv' gerado com sucesso.

Process finished with exit code 0
```